COLLEGE MINI-PROJECT REPORT

Department of Computer Engineering

Course: Da

1. Project Title

MergeFlow - File & Log Sorter Tool using Merge Sort and the Recursion Tree Method

2. Aim

To design, implement, and analyze an interactive, browser-based tool capable of sorting structured CSV records and chronological server log files using the top-down recursive Merge Sort algorithm, while visualizing the recursion tree structure, tracking algorithmic comparisons, and verifying theoretical time and space complexity.

3. Problem Statement

In real-world computing environments, debugging multi-service distributed architectures requires analyzing high-volume log streams and multi-column structured datasets. Standard browser tools or command-line scripts either sort data as a black box without exposing underlying algorithmic mechanics or fail to preserve multi-column row integrity when sorting by specific attributes.

Students and engineers need an educational yet practical tool that can:

1. Parse

4. Objectives

- Algorithm Fidelity: Implement an exact top-down recursive Merge Sort following academic reference standards (low >= high base case, integer floor midpoint split, recursive subproblem calls, and stable merging).

- Format

5. Hardware and Software Requirements

Hardware Requirements

- Processor: Modern multi-core x86-64 or ARM processor (Intel Core i3 / AMD Ryzen 3 / Apple Silicon or higher).
- RAM: M

Software Requirements

- Operating System: Platform-independent (Windows 10/11, macOS, Linux).
- Web B Brave).

6. Introduction to Merge Sort

Merge Sort is a quintessential divide-and-conquer comparison-based sorting algorithm invented by John von Neumann in 1945. It breaks a problem into smaller independent subproblems of identical structure, solves the subproblems recursively, and combines the results to solve the original problem.

Unlike Quick Sort - whose worst-case time degrades to O(n^2) when pivots are poorly selected - Merge Sort consistently delivers an optimal worst-case time complexity of O(n \log n). Furthermore, Merge Sort is naturally stable, meaning that elements with identical sorting keys preserve their relative order from the input, a crucial property when sorting multi-attribute databases.

7. Algorithm and Pseudocode

Academic Pseudocode (As per Reference Notes)

```
Algorithm MergeSort(arr, low, high):
    Input: Array arr, starting index low, ending index high
    Output: Subarray arr[low..high] sorted in non-decreasing order

    // Step 1: Base Case
    if low >= high then
        return

    // Step 2: Divide - Find the Middle
    mid = floor((low + high) / 2)

    // Step 3: Conquer - Recursively Sort Halves
    MergeSort(arr, low, mid)
    MergeSort(arr, mid + 1, high)

    // Step 4: Combine - Merge the Two Sorted Halves
    Merge(arr, low, mid, high)


Algorithm Merge(arr, low, mid, high):
    Input: Array arr with sorted subarrays arr[low..mid] and arr[mid+1..high]
    Output: Subarray arr[low..high] merged into sorted order

    n1 = mid - low + 1
    n2 = high - mid
    create temporary arrays L[0..n1-1] and R[0..n2-1]
```

```
    for i = 0 to n1 - 1 do
        L[i] = arr[low + i]
    for j = 0 to n2 - 1 do
        R[j] = arr[mid + 1 + j]

    i = 0, j = 0, k = low
    while i < n1 and j < n2 do
        increment comparison_count
        if compare(L[i], R[j]) <= 0 then
            arr[k] = L[i]
            i = i + 1
        else
            arr[k] = R[j]
            j = j + 1
        k = k + 1

    // Copy remaining elements
    while i < n1 do
        arr[k] = L[i]
        i = i + 1
        k = k + 1

    while j < n2 do
        arr[k] = R[j]
        j = j + 1
        k = k + 1
```

8. Recursive Splitting and Merging Process

The algorithm operates in two continuous phases:

1. Decon

2. Composition (Bottom-Up Merging):

Once the
using a two
prioritizes e

9. Recursion Tree Explanation

The Recursion Tree Method is an analytical and visual technique used to solve recurrence relations.

```
Level 0:                        [   T(n)   ]                    Work = c*n
                               /            \
Level 1:                 [ T(n/2) ]        [ T(n/2) ]           Work = 2 * c*(n/2) = c*n
                         /        \        /        \
Level 2:             [T(n/4)]  [T(n/4)]  [T(n/4)]  [T(n/4)]     Work = 4 * c*(n/4) = c*n
                       ...       ...       ...       ...
Level log?(n):       [T(1)] [T(1)] ... [T(1)] [T(1)]            Work = n * c*1 = c*n
```

Analysis:

- Tree Depth / Height: Because the problem size is halved at each step, the depth from root (depth 0) to leaf is log2 n.

- Cost p
level is

In MergeFlow, every node in the tree is interactively rendered with its recursion depth (D), index range, record count, and actual input values.

10. Time and Space Complexity

Metric	Complexity	Explanation
Best-Case Time	O(n \log n)	Splitting is unconditional; merging two lists
Average-Case Time	O(n \log n)	Expected case over random permutations; match
Worst-Case Time	O(n \log n)	Interleaved keys require the maximum n - 1 co
Auxiliary Space	O(n)	Merging requires buffer memory proportional t
Recursion Depth	O(\log n)	The balanced binary recursion tree reaches a

11. Description of Implemented Modules

1. User Interface Module (`index.html` CSS/DOM): Dark dashboard themed with green accents, housing the overview hero, metrics cards, upload dropzone, sort workbench, results table, interactive recursion tree SVG canvas, and activity log.

2. File I
automa

12. Application States and Screens

- State 1: Initial Empty / Demo Ready State: Displays dashboard overview with loaded sample data, ready metrics, and interactive controls.

- State
perform

13. Test Cases and Verification Results

#	Test Case Description	Input Data	Expected Result	Actual Result	Status
1	PDF Worked Numerical Example	`310, 285, 179, 652, 351, 423	179, 254, 285, 310, 351, 423, 450	Exactly match	PASSED

2	Already-Sorted Input	`10, 20, 30, 40, 50`	`10, 20, 30, 40, 50`	Preserved identical order with	**PASSED**
3	Reverse-Sorted Input	`50, 40, 30, 20, 10`	`10, 20, 30, 40, 50`	Correctly reversed to ascending	**PASSED**
4	Duplicate Values & Stability	`[Apple, 1], [Banana, 1], [Appl`	Apple rows must strictly preserve r	Apple items maintained exact	**PASSED**
5	Empty Input Handling	`""` (Empty string)	Safe exit, 0 comparisons, empty ta	Handled safely without runtime	**PASSED**
6	Single Element Base Case	`[42]`	Immediately returns; 0 comparison	Returned unchanged; 0 comp	**PASSED**
7	Negative and Decimal Numbers	`3.14, -10.5, 0, -2, 100.25, 2.718`	`-10.5, -2, 0, 2.718, 3.14, 100.25`	Sorted correctly by numeric va	**PASSED**
8	CSV Multi-column & Quoted Comm	`"Everything, Everywhere",20`	Rows intact, commas in quotes pre	Full rows preserved; quotes u	**PASSED**
9	Log Timestamps Chronological Sor	Out-of-order ISO server logs	Chronological timestamp order	Oldest logs placed first, line	**PASSED**
10	Descending Direction	`10, 50, 30, 20, 40` (descendi	`50, 40, 30, 20, 10`	Output accurately sorted high	**PASSED**
11	Metric Accuracy Verification	8 elements (n = 2^3)	Calls: 15, Merges: 7, Max Depth: 3	Calls: 15, Merges: 7, Max Dep	**PASSED**
12	Tree Node Count Check	n = 4 elements	Total nodes = 7, Root depth = 0	Exact tree matches 7 nodes,	**PASSED**
13	File Export Verification	Sample movies CSV & Run re	Valid RFC CSV and parsable JSON	Downloaded files valid and ma	**PASSED**
14	Malformed / Missing Keys	Mixed alphanumeric strings a	Handled using defined fallback wit	All records retained and sorte	**PASSED**
15	Large Dataset Scaling	1,000 randomized records	Sorted in <= 10 levels in < 500 ms	Sorted in 10 levels in ~ 22 ms	**PASSED**

14. Limitations

1. Single-Threaded Browser Scope: Merge Sort runs on the main browser thread. Files with hundreds of thousands of lines (> 50 \text{ MB}) can cause brief UI latency.
2. Visual visualiz

15. Conclusion

The MergeFlow File & Log Sorter Tool successfully fulfills all requirements of the Group G1 ISA assessment. The tool accurately implements top-down recursive Merge Sort strictly per reference academic notes, provides transparent empirical validation of O(n \log n) comparison scaling, and renders an interactive recursion tree method diagram. Its zero-dependency, fully local architecture guarantees effortless evaluation during laboratory assessments and presentations, while its GitHub, Vercel, and Render readiness demonstrates modern production standards.